

Date: _____

Lab # 04 TO UNDERSTAND AND OBSERVE THE WORKING AND RUNNING TIME OF HEAP SORT AND SHELL SORT ALGORITHM

Objectives

After completing this lab, students will be able to:

- Understand the working principles of **Heap Sort** and **Shell Sort**.
- Implement Heap Sort and Shell Sort using Python.
- Understand the concept of **Max Heap** and **Gap Sequence**.
- Trace the sorting process step-by-step.
- Analyze and compare the time and space complexities of both algorithms.
- Apply Heap Sort and Shell Sort to different input arrays.

Theory**1. Heap Sort****Definition**

Heap Sort is a **comparison-based sorting algorithm** that uses a binary heap data structure. It is based on the concept of a **Max Heap** for sorting elements in ascending order.

A **Max Heap** is a complete binary tree in which every parent node is greater than or equal to its children.

Heap Sort mainly consists of two steps:

1. Build a Max Heap from the given array.
2. Repeatedly remove the largest element from the heap and place it at the end of the array.

Working of Heap Sort

Suppose we have the following array:

[4, 10, 3, 5, 1]

Step 1: Build a Max Heap

The array is converted into a Max Heap:

```
    10
   / \
  5   3
 / \
4   1
```

Array representation:

[10, 5, 3, 4, 1]

Step 2: Swap the Root with the Last Element

The largest element, which is at the root, is swapped with the last element:

[1, 5, 3, 4, 10]

Now 10 is in its final sorted position.

Step 3: Heapify the Remaining Elements

The remaining elements are converted back into a Max Heap:

[5, 4, 3, 1, 10]

The process continues until all elements are sorted.

Final result:

[1, 3, 4, 5, 10]

Important Concepts

Parent and Child Indices

For an element at index i :

Left Child = $2*i + 1$

Right Child = $2*i + 2$

Parent = $(i-1) // 2$

These formulas are used when a heap is stored in an array.

Characteristics of Heap Sort

- **Time Complexity:** $O(n \log n)$ in best, average, and worst cases.
- **Space Complexity:** $O(1)$ auxiliary space.
- **In-place:** Yes.
- **Stable:** No.
- Uses a **Binary Heap** data structure.
- Suitable when guaranteed $O(n \log n)$ performance is required.

Applications

Heap Sort can be used when:

- Memory usage needs to be minimized.
- In-place sorting is required.
- Consistent $O(n \log n)$ worst-case performance is important.
- Priority-queue-related operations are required.

Heap Sort Algorithm

1. Build a Max Heap from the array.
 2. Set the heap size equal to the size of the array.
 3. Swap the first element with the last element.
 4. Reduce the heap size by one.
 5. Apply heapify to restore the Max Heap property.
 6. Repeat steps 3–5 until the heap contains one element.
 7. The array is now sorted.
-

Heap Sort Python Code

```
def heapify(arr, n, i):

    largest = i

    left = 2 * i + 1
    right = 2 * i + 2

    # Check if left child is greater than root
    if left < n and arr[left] > arr[largest]:
        largest = left

    # Check if right child is greater than largest
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Recursively heapify the affected subtree
        heapify(arr, n, largest)

def heap_sort(arr):

    n = len(arr)

    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements from heap
    for i in range(n - 1, 0, -1):

        # Move largest element to the end
        arr[0], arr[i] = arr[i], arr[0]

        # Heapify the remaining heap
        heapify(arr, i, 0)
```

```
# Example
array = [4, 10, 3, 5, 1]

heap_sort(array)

print("Sorted array:", array)
Output
Sorted array: [1, 3, 4, 5, 10]
```

2. Shell Sort

Definition

Shell Sort is an **in-place comparison-based sorting algorithm** that is an improved version of Insertion Sort.

Instead of comparing adjacent elements only, Shell Sort initially compares elements that are far apart using a **gap**.

The gap is gradually reduced until it becomes 1. When the gap is 1, Shell Sort performs a normal insertion-sort-like operation.

Working of Shell Sort

Consider:

[14, 10, 29, 16, 12, 32, 13, 35, 2, 40, 1]

The initial gap is:

$gap = n // 2$

Since:

$n = 11$

we get:

$gap = 5$

Elements separated by 5 positions are compared and rearranged.

After that:

$gap = 5 // 2 = 2$

Then:

$gap = 2 // 2 = 1$

Finally, when:

gap = 1

the array is sorted using the insertion-sort process.

Gap Sequence

For an array of size 11:

11 → 5 → 2 → 1

The exact performance of Shell Sort depends significantly on the selected gap sequence.

Characteristics of Shell Sort

- **In-place:** Yes.
- **Stable:** No.
- Requires very little additional memory.
- Improves upon traditional Insertion Sort by comparing elements that are far apart.
- Performance depends on the gap sequence.
- Worst-case complexity depends on the gap sequence used.

Applications

Shell Sort can be useful for:

- Moderately sized datasets.
- Situations where extra memory should be minimized.
- Arrays that are partially sorted.
- Applications where a simple in-place sorting algorithm is required.

Shell Sort Algorithm

1. Set gap = n // 2.
2. Compare elements that are gap positions apart.
3. Perform insertion-sort-like shifting for these elements.
4. Reduce the gap by half.
5. Repeat the process.
6. Continue until gap becomes 1.
7. The array is completely sorted.

Shell Sort Python Code

```
def shell_sort(arr):
```

```
    n = len(arr)
```

```
    # Start with half of the array size
```

```
gap = n // 2

while gap > 0:

    # Perform insertion sort with the current gap
    for i in range(gap, n):

        temp = arr[i]
        j = i

        # Shift elements that are greater than temp
        while j >= gap and arr[j - gap] > temp:

            arr[j] = arr[j - gap]
            j -= gap

        # Place temp at its correct position
        arr[j] = temp

    # Reduce the gap
    gap //= 2

# Example
array = [14, 10, 29, 16, 12, 32, 13, 35, 2, 40, 1]

shell_sort(array)

print("Sorted array:", array)
Output
Sorted array: [1, 2, 10, 12, 13, 14, 16, 29, 32, 35, 40]
```

Exercise:

Tasks

1. Implement the Heap Sort algorithm in Python and sort the given array in ascending order: [25, 17, 31, 13, 2, 8, 19].
2. Trace the Heap Sort algorithm for the array [4, 10, 3, 5, 1] by showing the Max Heap construction, swapping, heapifying, and final sorted array.
3. Implement the Shell Sort algorithm in Python and sort the given array in ascending order: [14, 10, 29, 16, 12, 32, 13, 35, 2, 40, 1].
4. Trace the Shell Sort algorithm for the array [20, 7, 15, 3, 12, 10] by showing the array after each gap value until the final sorted array is obtained.
5. Compare Heap Sort and Shell Sort in terms of working principle, time complexity, space complexity, stability, and whether they are in-place sorting algorithms.